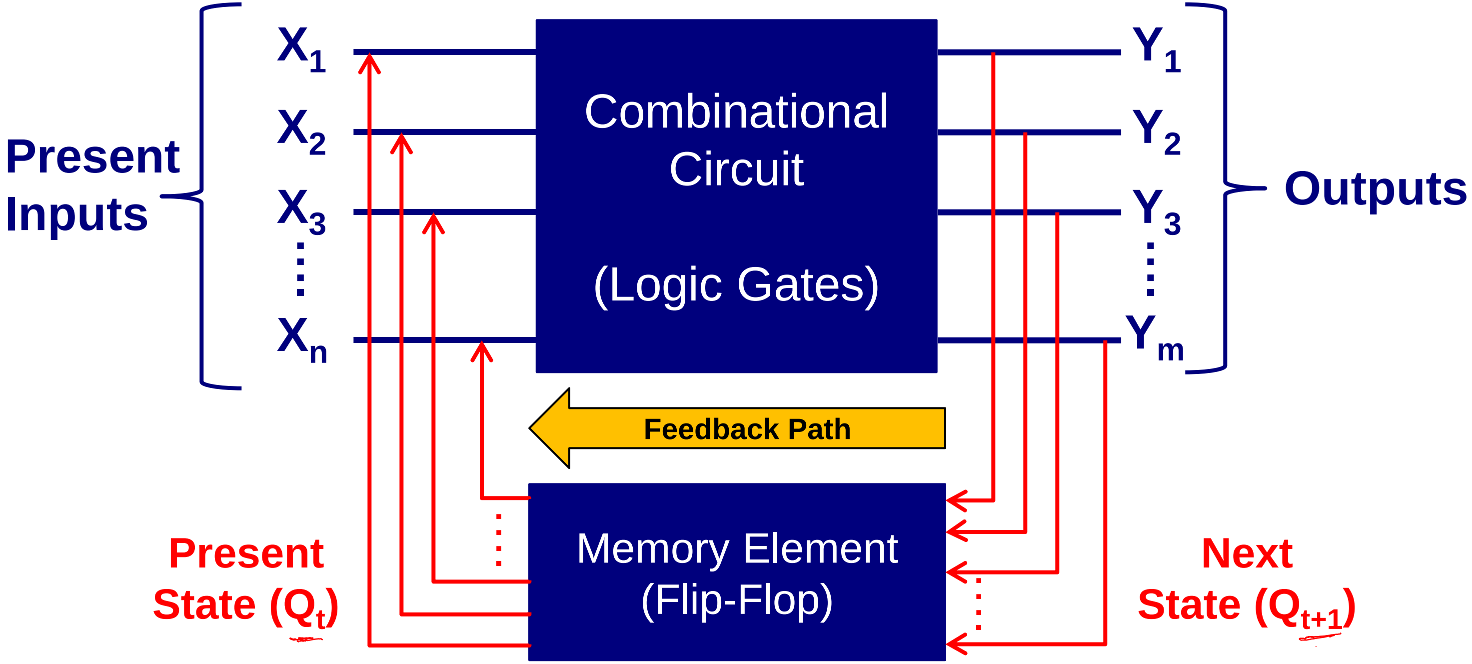
Sequential Circuits

Dr. Nilesh Patidar and Shiraz Husain

Sequential Circuit



Does contain memory element/feedback path

Combinational v/s Sequential

Combinational Circuits

- Outputs depend only on present inputs.
- Feedback path is not present.
- Memory elements are not required.
- Clock signal is not required.
- Easy to design.

Sequential Circuits

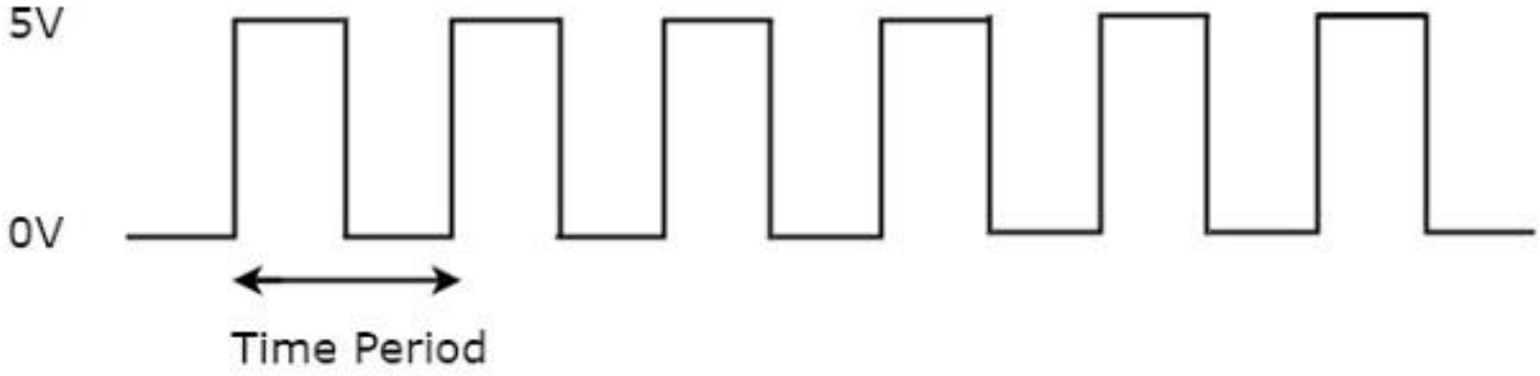
- Outputs depend on both present inputs and present state.
- Feedback path is present.
- Memory elements are required.
- Clock signal is required.
- Difficult to design.

Types of Sequential Circuits

- Following are the two types of sequential circuits −
  - Asynchronous sequential circuits
  - Synchronous sequential circuits
- Asynchronous sequential circuits
  - If the output of a sequential circuit depend on active transition of input, then it is called as **Asynchronous sequential circuit**.
- Synchronous sequential circuits
  - If the output of a sequential circuit depend on active transition of clock, then it is called as **synchronous sequential circuit**.

Clock signal

- Clock signal is a periodic signal and its ON time and OFF time need not be the same. We can represent the clock signal as a **square wave.**

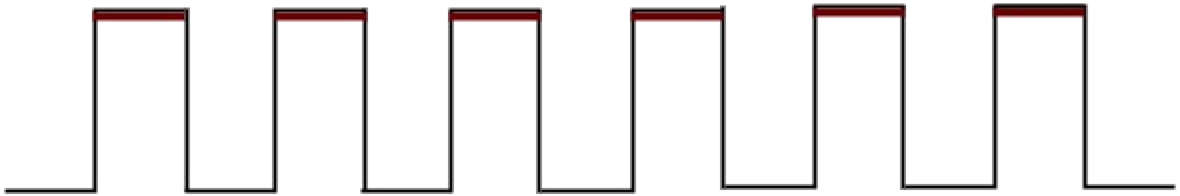


Types Triggering

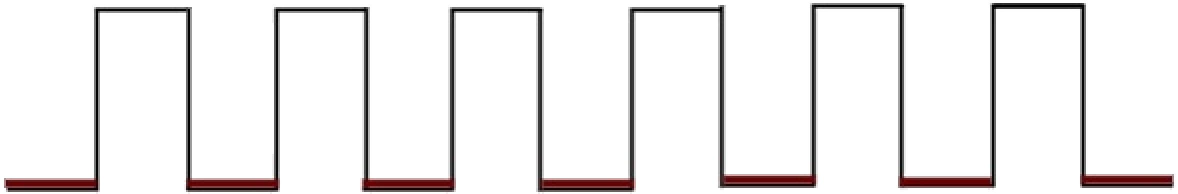
- Following are the two possible types of triggering in Sequential Circuits-
  - Level triggering
    - Positive Level
    - Negative Level
  - Edge triggering
    - Positive Edge
    - Negative Edge

Level triggering

- If the sequential circuit is operated with the clock signal when it is in **Logic High**, then that type of triggering is known as **Positive level triggering**.

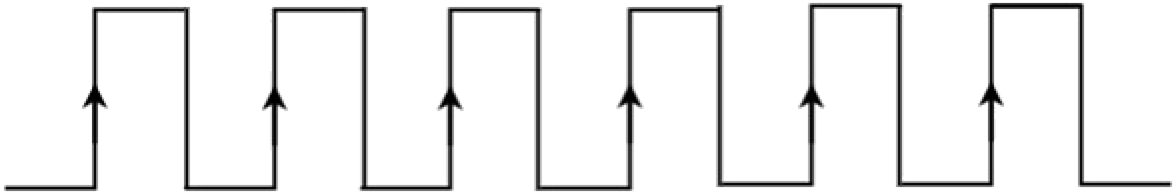


- If the sequential circuit is operated with the clock signal when it is in **Logic Low**, then that type of triggering is known as **Negative level triggering**.

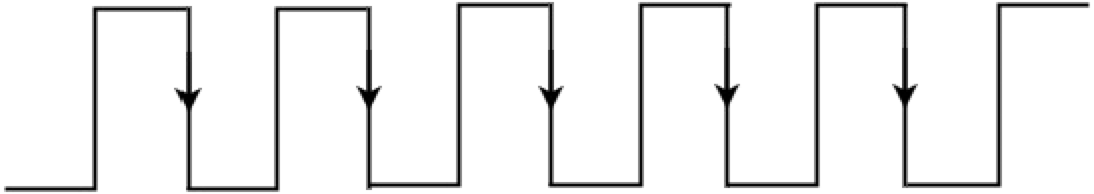


Edge triggering

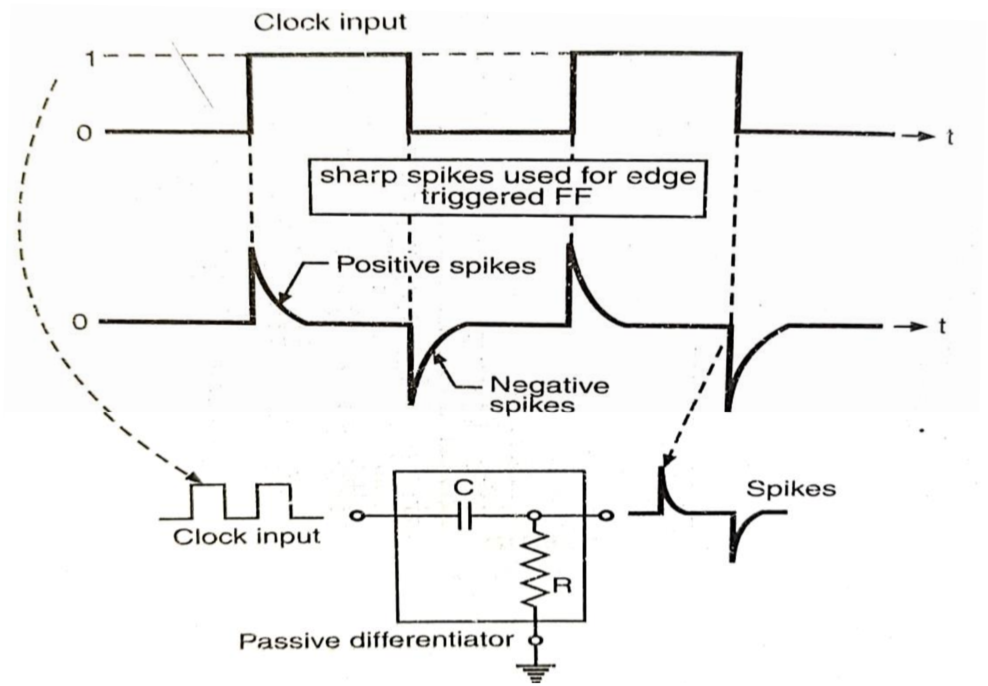
- If the sequential circuit is operated with the clock signal that is transitioning from Logic Low to High, then that type of triggering is known as **Positive edge triggering**.



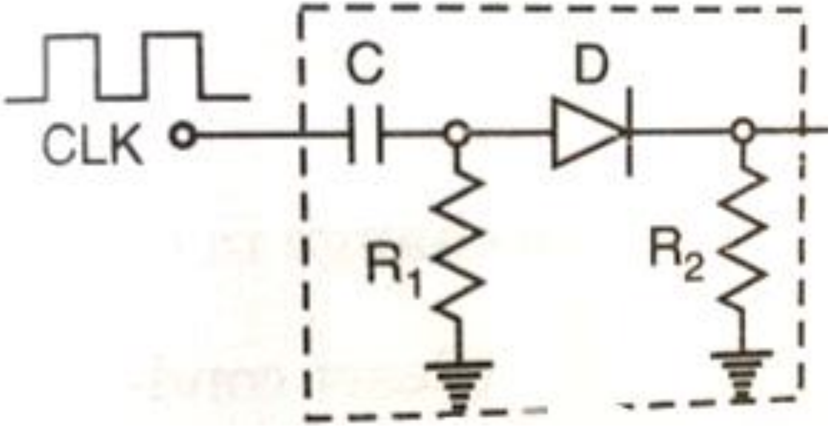
- If the sequential circuit is operated with the clock signal that is transitioning from Logic High to Low, then that type of triggering is known as **Negative edge triggering**.



Conversion of Level to Edge Trigerring



Conversion of Level to Positive Edge Trigerring

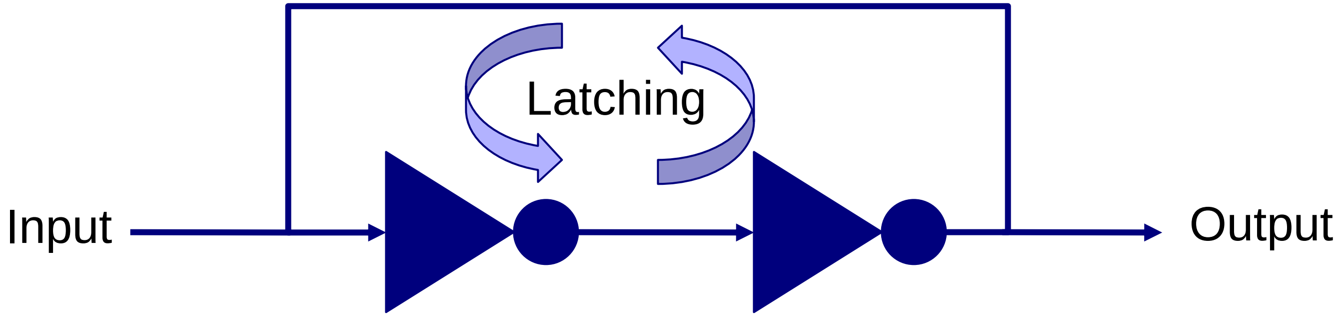


Memory Elements (Latch or Flip-flop)

- There are two types of memory elements based on the type of triggering that is suitable to operate it.
  - Latches
  - Flip-flops
- Latches operate with enable signal, which is **level sensitive** *(level triggering).*
- Flip-flops operate with clock signal, which is **edge sensitive** *(edge triggering).*

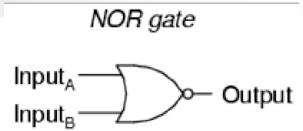
Basic Latch

- A latch is a device used to hold the digital information.
- The basic latch can be designed using 2 inverters.



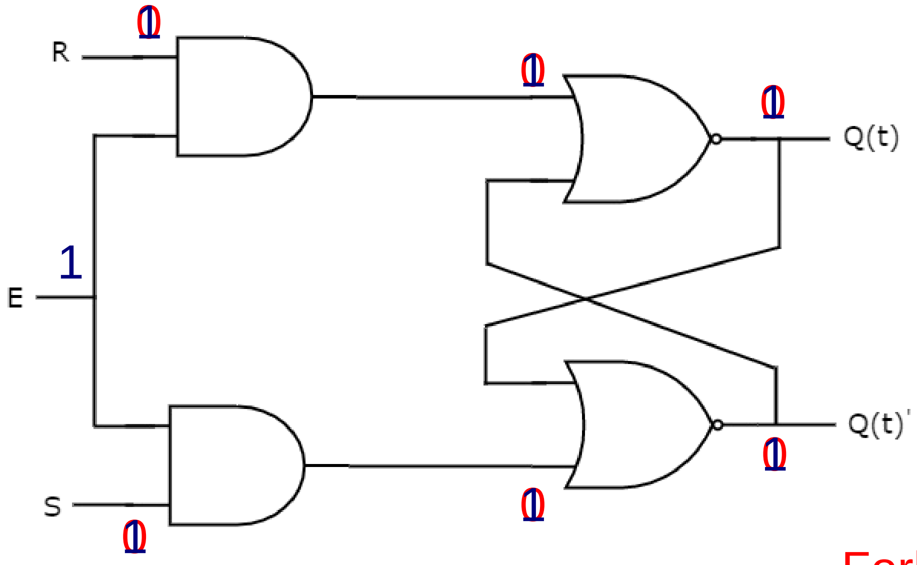
- Latch means store the information.

SR Latch (NOR)



A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

- SR Latch is also called as **Set Reset Latch**. This latch affects the outputs as long as the enable, E is maintained at '1'



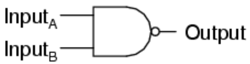
State Table

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	-

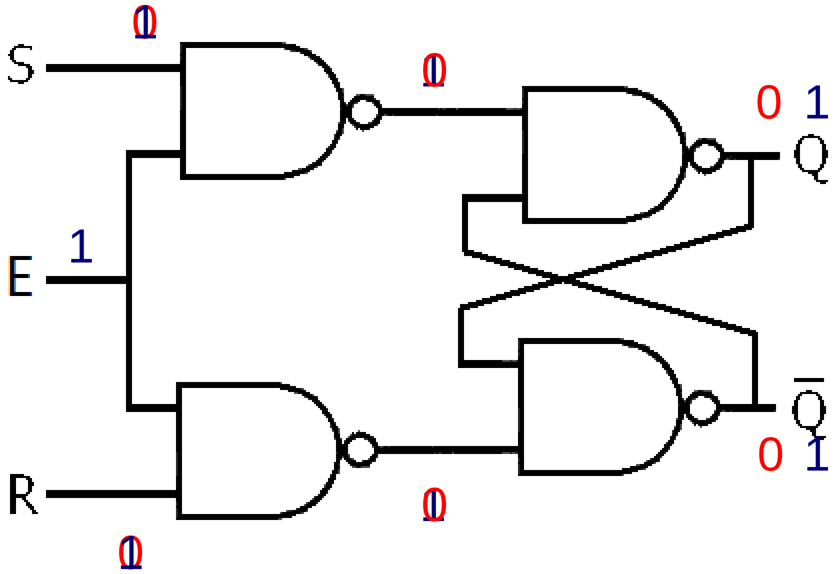
Forbidden State

SR Latch (NAND)

NAND gate



A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



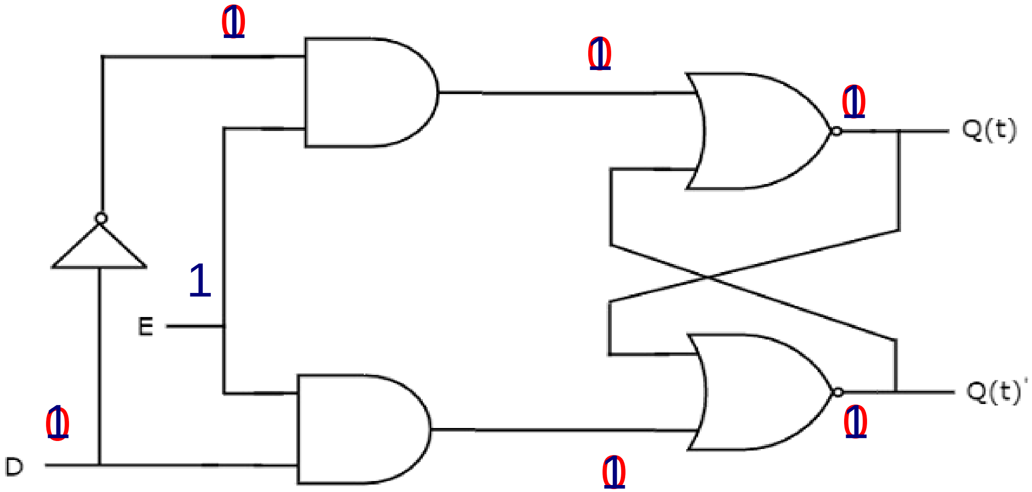
State Table

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	-

Forbidden State

D Latch

- There is one drawback of SR Latch. That is the next state value can't be predicted when both the inputs S & R are one. So, we can overcome this difficulty by D Latch. It is also called as **Data Latch.**



State Table

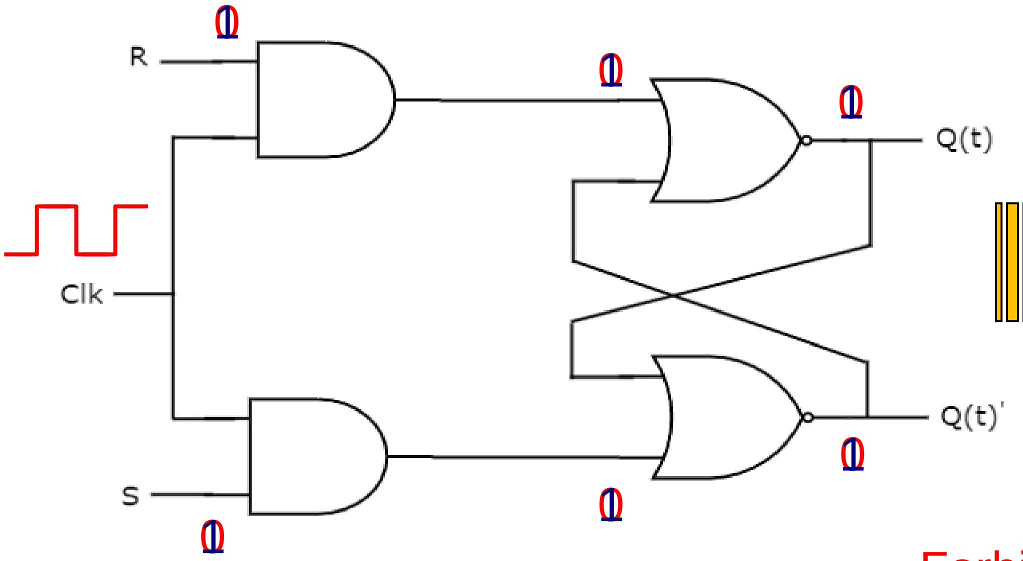
D	Q_{t+1}
0	0
1	1

Flip-Flops

- by **cascade two latches** in such a way that the first latch is enabled for every positive clock pulse and second latch is enabled for every negative clock pulse. So that the combination of these two latches become a flip-flop.
- There are four types of flip-flops.
  - SR Flip-Flop
  - D Flip-Flop
  - JK Flip-Flop
  - T Flip-Flop

SR Flip-flop

- SR flip-flop operates with only positive clock transitions or negative clock transitions.



State Table

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	-

Forbidden State

SR Flip-flop

Characteristic Table

Present Input		Present State	Next State	Status
S	**R**	Q_t	Q_{t+1}	
0	0	0	0	No Change
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	X	Invalid
1	1	1	X	

Excitation Table

Present State	Next State	Present Input	
Q_t	Q_{t+1}	**S**	**R**
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

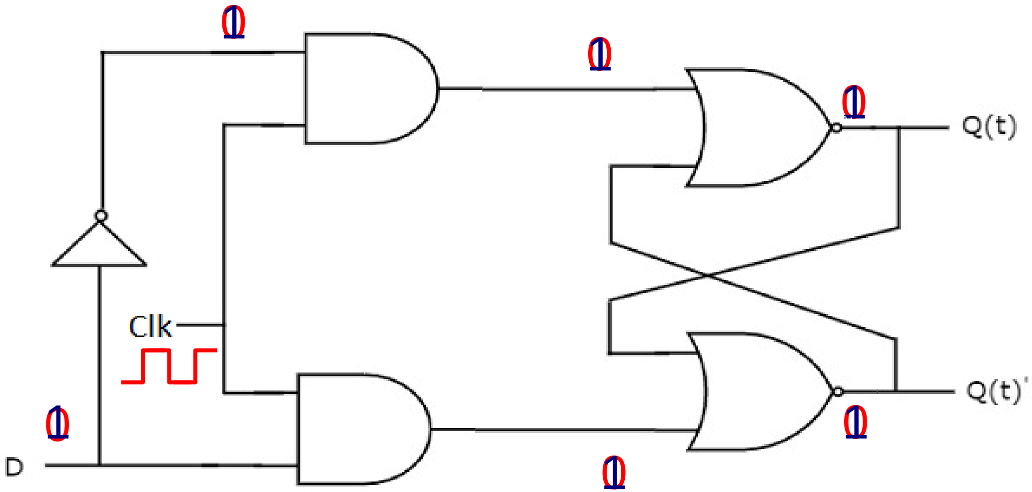
SR Flip-flop

K-map for Q_{t+1}

Q_{t+1} S \ RQ_t	$R'Q_t'$ (00)	$R'Q_t$ (01)	RQ_t (11)	RQ_t' (10)
S' (0)	0 0	1 1	0 3	0 2
S (1)	1 4	1 5	X 7	X 6

$$Q_{t+1} = S + R'Q_t$$

D Flip-flop



State Table

D	Q_{t+1}
0	0
1	1

D Flip-flop

Characteristic Table

Present Input	Present State	Next State	Status
D	Q_t	Q_{t+1}	
0	0	0	Reset
0	1	0	
1	0	1	Set
1	1	1	

Excitation Table

Present State	Next State	Present Input
Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

D Flip-flop

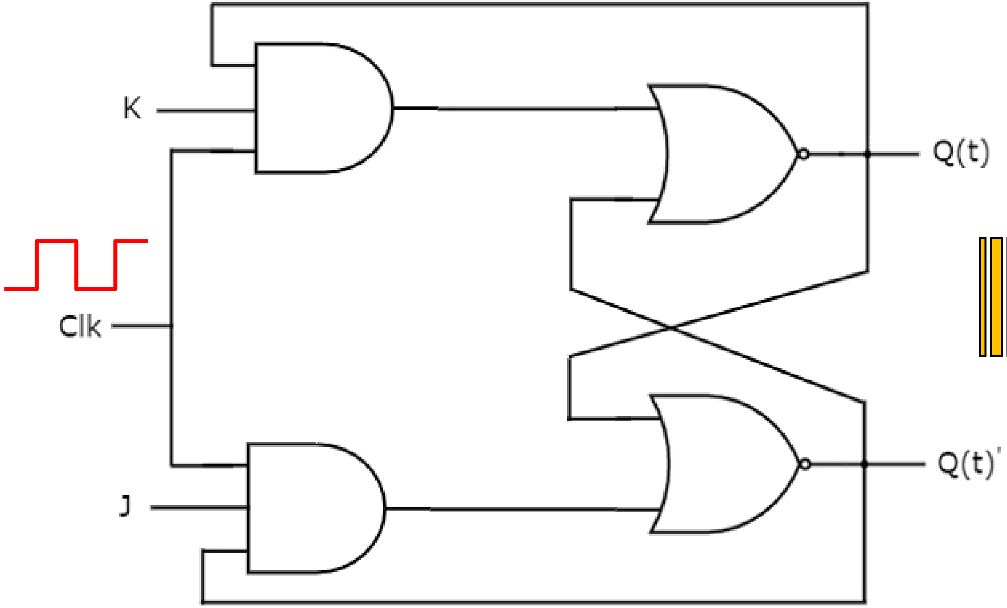
K-map for Q_{t+1}

Q_{t+1} — D \ Q_t	Q_t' (0)	Q_t (1)
D' (0)	0 0	0 1
D (1)	1 2	1 3

$$Q_{t+1} = D$$

JK Flip-flop

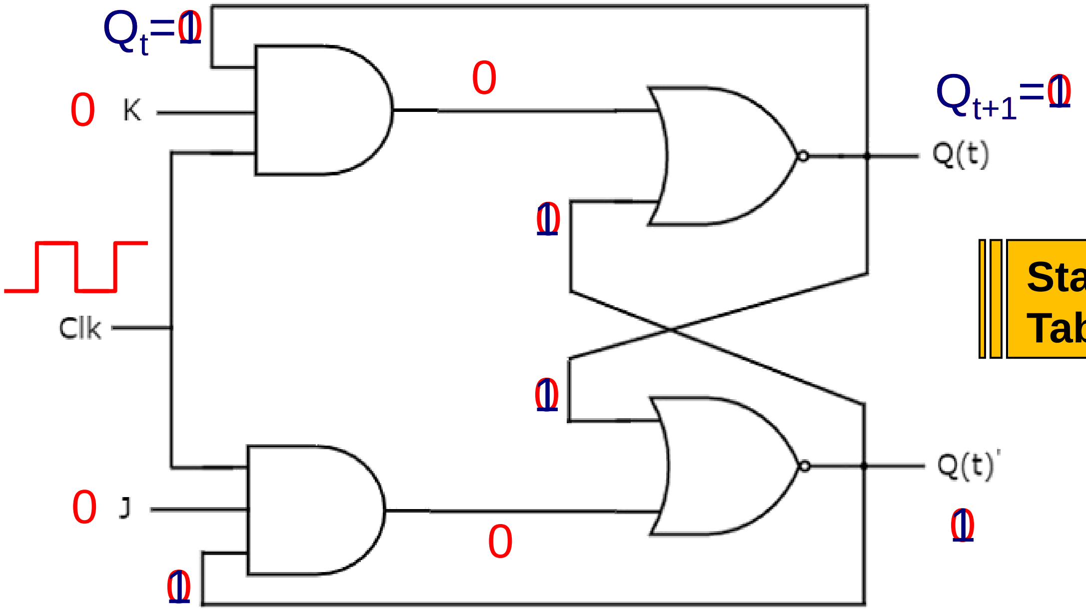
- JK flip-flop is the modified version of SR flip-flop..



State Table

J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

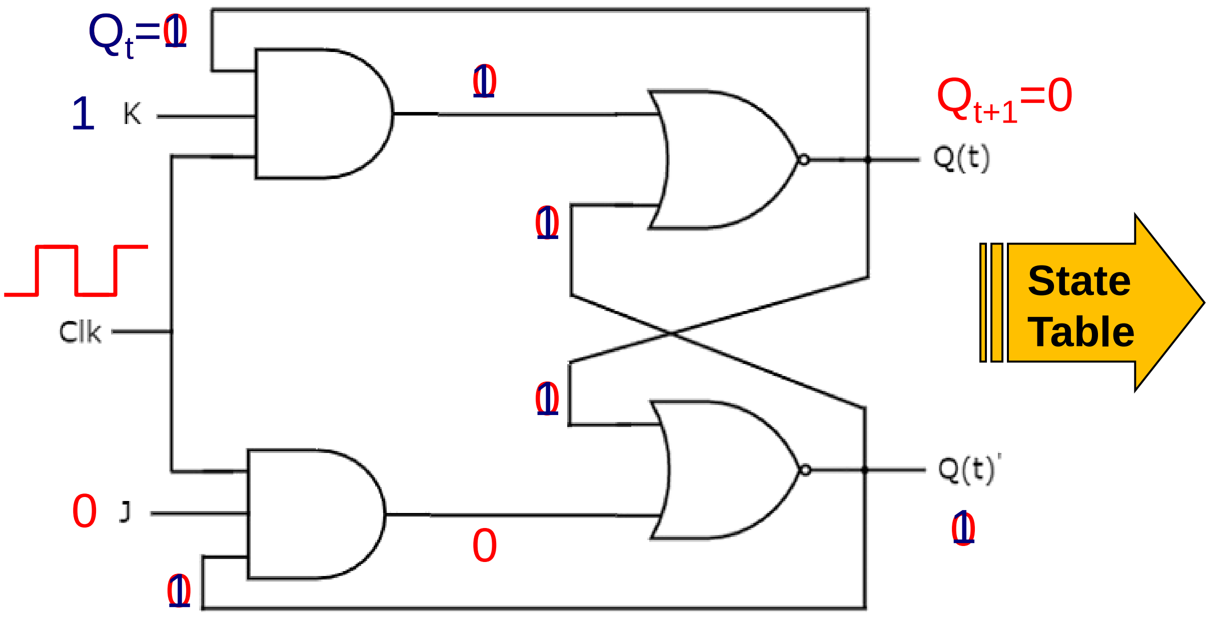
JK Flip-flop



State Table

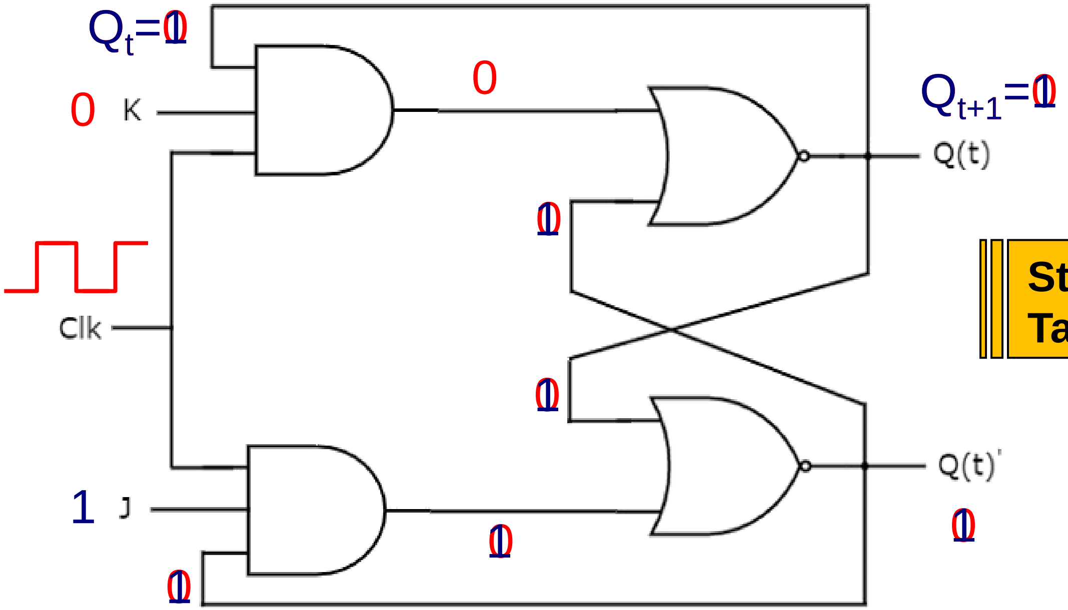
J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

JK Flip-flop



J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

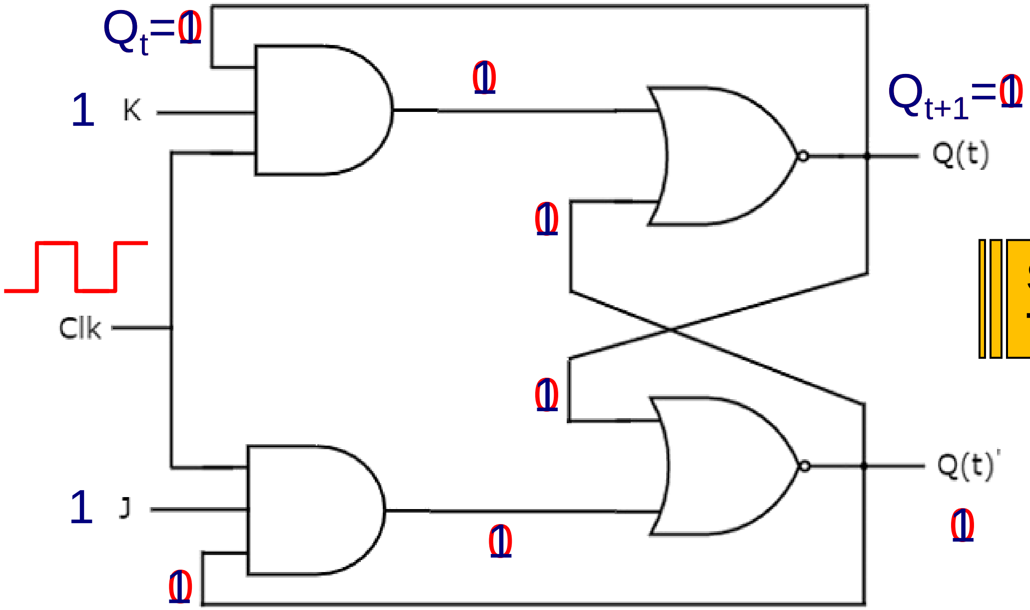
JK Flip-flop



State Table

J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

JK Flip-flop



State Table

J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	Q_t'

JK Flip-flop

Characteristic Table

Present Input		Present State	Next State	Status
J	K	Q_t	Q_{t+1}	
0	0	0	0	No Change
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	1	Toggle
1	1	1	0	

Excitation Table

Present State	Next State	Present Input	
Q_t	Q_{t+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

JK Flip-flop

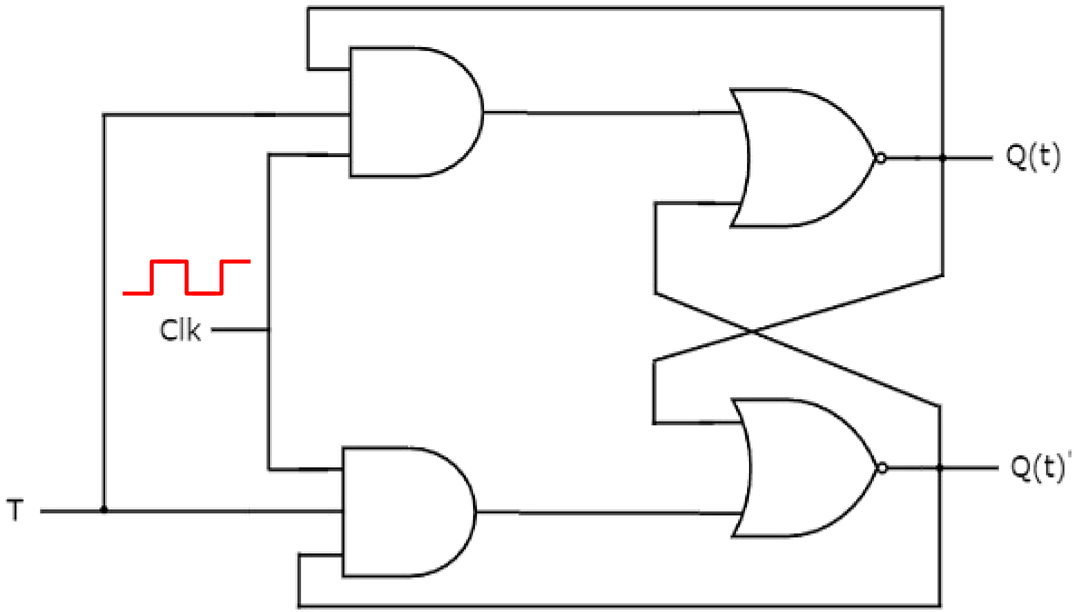
K-map for Q_{t+1}

Q_{t+1} J \ KQ_t	$K'Q_t'$ (00)	$K'Q_t$ (01)	KQ_t (11)	KQ_t' (10)
J' (0)	0 0	1 1	0 3	0 2
J (1)	1 4	1 5	0 7	1 6

$$Q_{t+1} = JQ_t' + K'Q_t$$

T Flip-flop

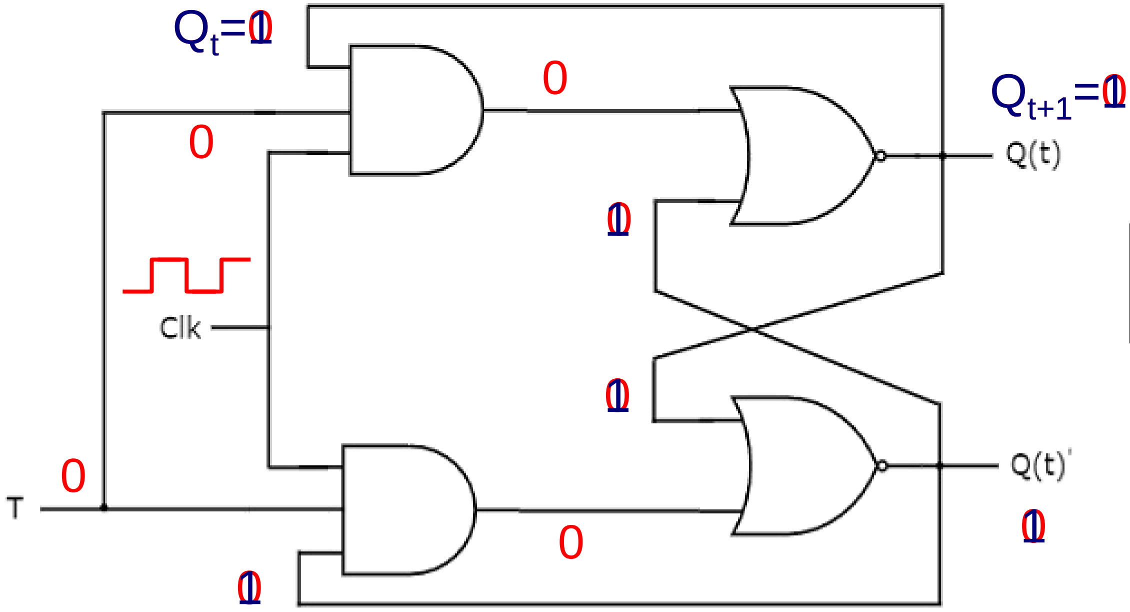
- We can implement T Flip-flop by joining the inputs JK flip-flop.



State Table

T	Q_{t+1}
0	Q_t
1	Q_t'

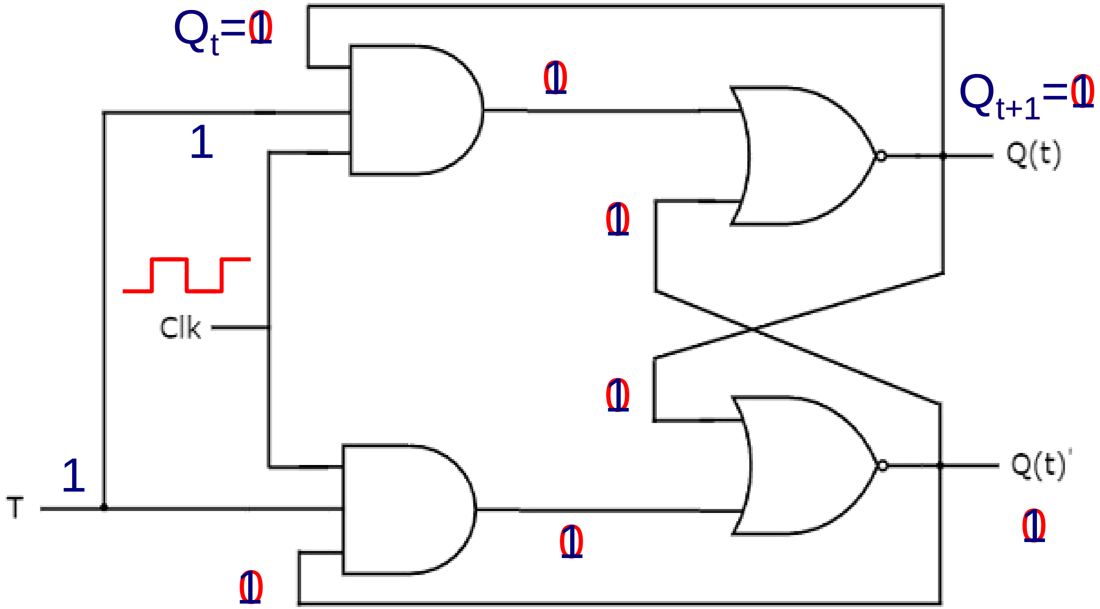
T Flip-flop



State Table

T	Q_{t+1}
0	Q_t
1	Q_t'

T Flip-flop



State Table

T	Q_{t+1}
0	Q_t
1	Q_t'

T Flip-flop

Characteristic Table

Present Input	Present State	Next State	Status
T	Q_t	Q_{t+1}	
0	0	0	No Change
0	1	1	
1	0	1	Toggle
1	1	0	

Excitation Table

Present State	Next State	Present Input
Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

T Flip-flop

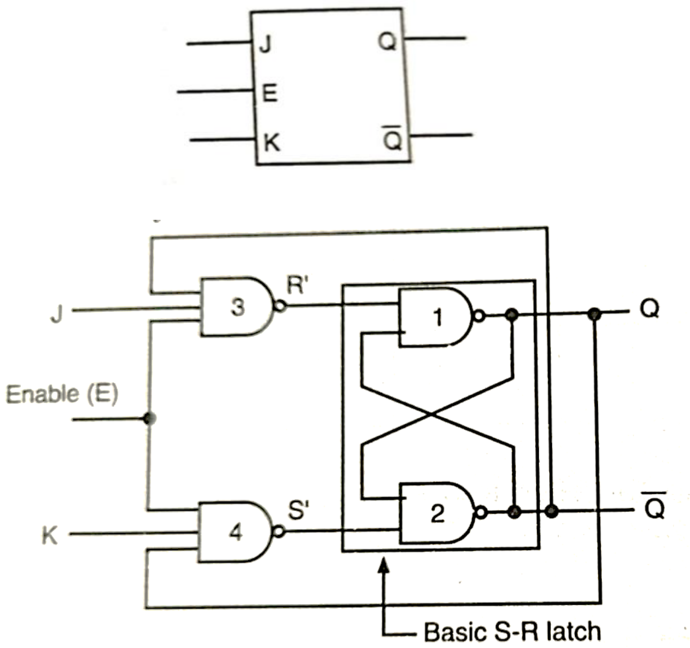
K-map for Q_{t+1}

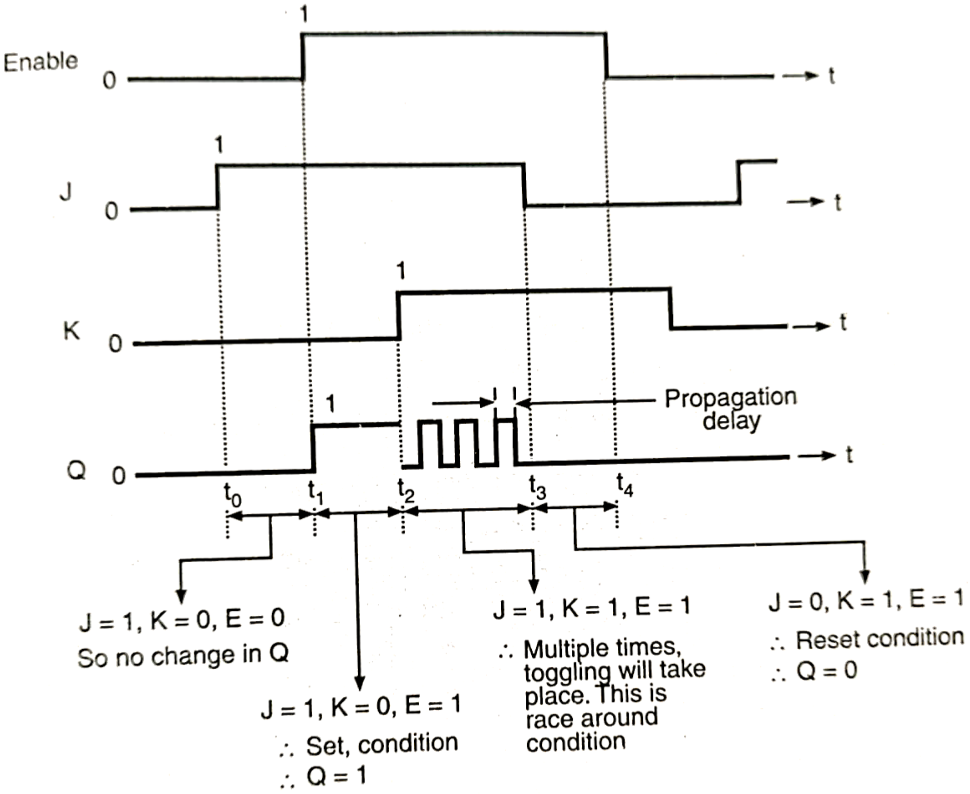
Q_{t+1} — $T \backslash Q_t$	Q_t' **(0)**	Q_t **(1)**
T' (0)	0 **0**	1 **1**
T (1)	1 **2**	0 **3**

$$Q_{t+1} = TQ_t' + T'Q_t$$

$$Q_{t+1} = T \oplus Q_t$$

Race Around Condition





How to avoid Race Around Condition

- The race around condition in JK Latch can be avoided by
  - Using the edge triggered JK flip flop
    - For the race around to take place, it is essential to have the enable input high along with J=K=1. As the enable input remains high for a long time in a JK latch, the problem of multiple toggling arises. But in edge triggered JK flip flop, the positive clock pulse is present only for a very short time. Hence, by the time the changed outputs returns to the inputs of NAND gates, the clock pulse has died down to zero. Hence the multiple toggling cannot take place. Thus, edge triggering avoids the race around condition.
  - Using the Master Slave JK flip flop

Master-Slave JK Flip-flop

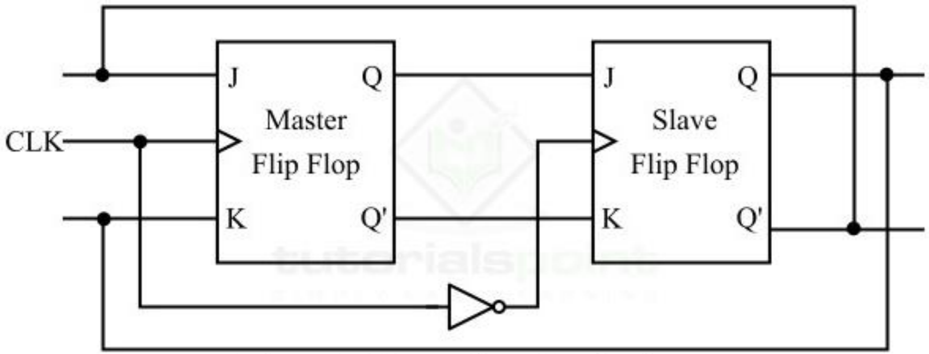


Figure 1 - Master-Slave JK Flip Flop

- In this combination of two JK flip flop, one acts as a **master flip flop** and the other acts as a **slave flip flop**. In this master-slave flip flop, the outputs of the master JK flip flop are connected to the inputs of the slave JK flip flop. The outputs of the slave flip flop are fed back to the inputs of the master JK flip flop.
- In the master-slave JK flip flop, a NOT gate (Inverter) is also used which is connected to clock signal to provide inverted clock signal to the slave flip flop.
- Therefore, when clock signal to master flip flop is 0, then for slave flip flop the clock signal is 1 and vice-versa

Operation of Master-Slave JK Flip Flop

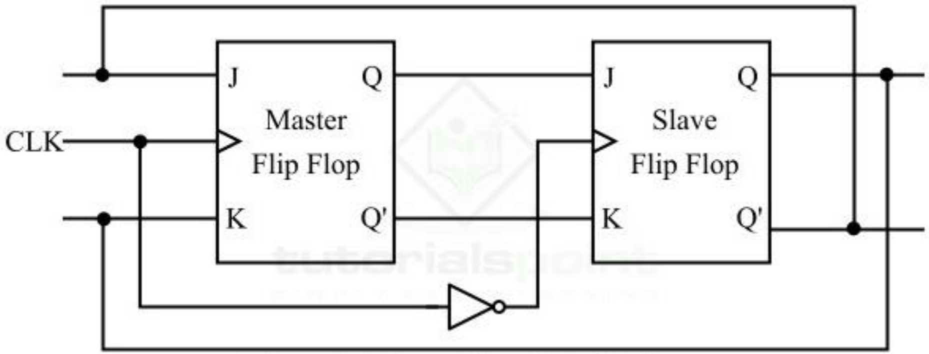
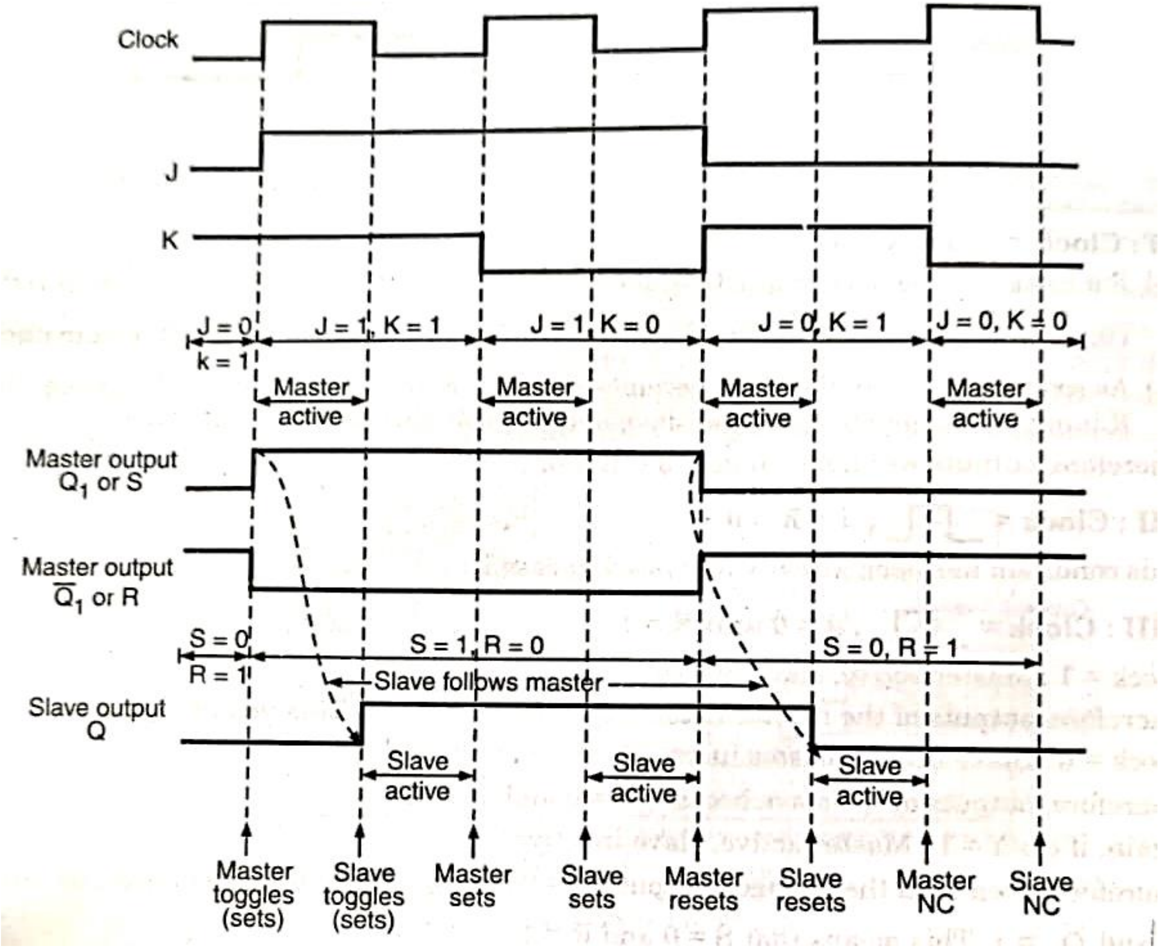


Figure 1 - Master-Slave JK Flip Flop

- When the clock pulse goes to high, the slave flip flop becomes inactive and the inputs J and K can control the state of the system.
- When the clock pulse goes back to low, the information is transferred from master flip flop to the slave flip flop, and the final output of the system is obtained.
- From the circuit, it is clear that the master flip flop is positive level triggered and the slave flip flop is negative level triggered. Consequently, the master flip flop responds before the slave flip flop. Now, let us discuss the operation of the master-slave JK flip flop for different combinations of inputs J and K

Waveform

- Observations:
- The slave always follows the master, after a delay of half clock cycle period.
- The multiple toggling or the race around condition is avoided.



Nilesh Patidar and Shiraz Husain

Conversion of Flip-Flops

- The **excitation table** for all flip-flops is shown below.

Present State	Next State	SR-FF		D-FF	JK-FF		T-FF
Q_t	Q_{t+1}	S	R	D	J	K	T
0	0	0	X	0	0	X	0
0	1	1	0	1	1	X	1
1	0	0	1	0	X	1	1
1	1	X	0	1	X	0	0

Conversion of Flip-Flops

- We can convert one flip-flop into the remaining three flip-flops by including some additional logic. So, there will be total of twelve **flip-flop conversions**.
- Follow these **steps** for converting one flip-flop to the other.
  - Step1: Create a clubbed excitation table of both given and required flip flop.
  - Step 2: Remove the next state column for the table.
  - Step3: Get the **simplified expressions** for each excitation input. If necessary, use K-maps for simplifying.
  - Step4: Draw the **circuit diagram** of desired flip-flop according to the simplified expressions using given flip-flop and necessary logic gates.

Conversion of SR-FF to D-FF

- Step1: Create a clubbed excitation table of both given and required flip flop.
- Step 2: Remove the next state column for the table.

Present Input	Present State	Next State	Excitation Inputs	
D	**Q_t**	**Q_{t+1}**	**S**	**R**
0	0	0	0	X
1	0	1	1	0
0	1	0	0	1
1	1	1	X	0

Conversion of SR-FF to D-FF

- Step3: Get the **simplified expressions** for each excitation input. If necessary, use K-maps for simplifying.

Present Input	Present State	Excitation Inputs	
D	**Q_t**	**S**	**R**
0	0	0	X
1	0	1	0
0	1	0	1
1	1	X	0

K-map for S

S

D \ Q_t	Q_t' (0)	Q_t (1)
D' (0)	0 [0]	0 [1]
D (1)	1 [2]	X [3]

S = D

Conversion of SR-FF to D-FF

- Step3: Get the **simplified expressions** for each excitation input. If necessary, use K-maps for simplifying.

Present Input	Present State	Excitation Inputs	
D	**Q_t**	**S**	**R**
0	0	0	X
1	0	1	0
0	1	0	1
1	1	X	0

K-map for R

R: **D** \ **Q_t**	**Q_t' (0)**	**Q_t (1)**
D' (0)	X (0)	1 (1)
D (1)	0 (2)	0 (3)

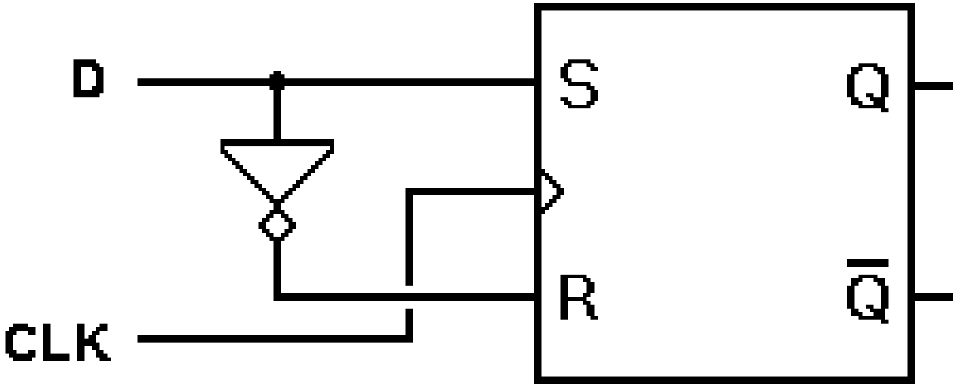
R = D'

Conversion of SR-FF to D-FF

- Step 4: Draw the **circuit diagram** of desired flip-flop according to the simplified expressions using given flip-flop and necessary logic gates.

S = D

R = D’



Conversion of D-FF to T-FF

- Step 1: Create a clubbed excitation table of both given and required flip flop.
- Step 2: Remove the next state column for the table.

Present Input	Present State	Next State	Excitation Inputs
T	**Q_t**	**Q_{t+1}**	**D**
0	0	0	0
1	0	1	1
1	1	0	0
0	1	1	1

Conversion of D-FF to T-FF

- Step3: Get the **simplified expressions** for each excitation input. If necessary, use K-maps for simplifying.

Present Input	Present State	Excitation Inputs
T	**Q_t**	**D**
0	0	0
1	0	1
1	1	0
0	1	1

K-map for D

D (T \ Q_t)	**Q_t' (0)**	**Q_t (1)**
T' (0)	0 $_{0}$	1 $_{1}$
T (1)	1 $_{2}$	0 $_{3}$

$$D = TQ_t' + T'Q_t$$

$$D = T \oplus Q_t$$

Conversion of D-FF to T-FF

- Step 4: Draw the **circuit diagram** of desired flip-flop according to the simplified expressions using given flip-flop and necessary logic gates.

$$D = TQ_t' + T'Q_t$$

$$D = T \oplus Q_t$$